

05 刷题模式与模板

目标:看完能直接抄 ACM 模式骨架 + 10 大算法模板上战场。

重点:模板给到,原理点到为止(不展开算法讲解)。

平台:ACM 模式(牛客)+ LeetCode 模式。

C++ 类比:本篇主要把 C++ 模板翻译成 Java 模板。

一、开篇定调:这一章是干什么的

Java 刷题 70% 的时间花在两件事上:模式骨架 和 算法模板。

- 模式骨架:牛客要自己写 main + throws IOException + BufferedReader(见 01);LeetCode 只写一个 class Solution 里的方法。两种模式结构完全不同,切平台必踩坑。

- 算法模板:看完 03 容器、04 工具类后,真正上战场就要靠 10 大算法的完整可抄模板。这一章不展开算法原理(那是一整本书),只给能直接抄的代码 + 思路一句话。

读完这一章,你的预期是:

- 看到 LeetCode 题目 → 30 秒判断属于哪类算法 → 抄对应模板 → 改 1-2 个变量 → 提交
- 看到牛客题目 → 直接套 ACM 骨架,把算法填进去

下面所有模板,只要带 class Solution { ... } 的,复制到 LeetCode 就能提交;带 class Main 的,复制到牛客就能跑。

二、ACM 模式完整骨架(牛客 / 笔试)

详细 I/O 选型见 01。本节只给"主方法完整骨架",I/O 选型按 01 推荐(StreamTokenizer + PrintWriter)。

2.1 三种输入模式全骨架(单组 / 多组 T / 哨兵 0)

模式 A:单组输入(只跑一次业务)

```
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StreamTokenizer in = new StreamTokenizer(br);
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));

        // ===== 业务逻辑写在这里 =====
        in.nextToken();
        int n = (int) in.nval;    // 读第一个整数
        // ... 自己的逻辑 ...
        out.println(ans);

        out.flush();            // 必须有,否则可能丢输出
    }
}
```

模式 B:多组 T 输入(第一行给 T,后面 T 组)

```
import java.io.*;
import java.util.*;

public class Main {
    public static void main(String[] args) throws IOException {
        StreamTokenizer in = new StreamTokenizer(new BufferedReader(new InputStreamReader(System.in)));
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));
        StringBuilder sb = new StringBuilder();

        in.nextToken();
        int T = (int) in.nval;    // 第一行:组数
        for (int t = 0; t < T; t++) {
            // ===== 每组的输入 + 业务逻辑 =====
            in.nextToken();
            int a = (int) in.nval;
            in.nextToken();
            int b = (int) in.nval;
            sb.append(a + b).append("\n");
        }

        out.print(sb);
        out.flush();
    }
}
```

模式 C:哨兵 0 0 终止(不告诉 T,读到 0 0 结束)

```
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        StreamTokenizer in = new StreamTokenizer(new BufferedReader(new InputStreamReader(System.in)));
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));
        StringBuilder sb = new StringBuilder();

        while (true) {
            in.nextToken();
            int a = (int) in.nval;
            in.nextToken();
            int b = (int) in.nval;
            if (a == 0 && b == 0) break; // 哨兵条件,具体看题
            sb.append(a + b).append("\n");
        }

        out.print(sb);
        out.flush();
    }
}
```

2.2 关键三件套(必背)



代码片段	作用	背下来
<code>`throws IOException`</code>	加在 `main` 后面,声明可能抛 I/O 异常	必须
<code>`StreamTokenizer in = new StreamTokenizer(new BufferedReader(new InputStreamReader(System.in)));`</code>	高速读入器	必须
<code>`PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));`</code>	高速输出器	必须
<code>`out.flush();`</code>	强制刷新缓冲区,大输出不写可能丢	必须

2.3 C++ 对照(感受 Java 的"仪式感")

```
#include <iostream>
int main() {
    int n; std::cin >> n;    // Java: in.nextToken(); int n = (int) in.nval;
    int ans; /* 业务 */      // 业务逻辑一样
    std::cout << ans << std::endl; // Java: out.println(ans); out.flush();
    return 0;
}
```

核心差异:Java 比 C++ 多 4 行"仪式代码"(import + throws + StreamTokenizer 构造 + flush)。这是固定开销,每个 ACM 题目都一样,抄就完事。

三、LeetCode 模式完整骨架

3.1 类结构差异(对比 ACM)

项	ACM 模式(牛客)	LeetCode 模式
类名	<code>`public class Main`</code>	<code>`public class Solution`</code>
<code>`main`</code>	必须写	不写
异常声明	<code>`throws IOException`</code>	不写
<code>`import`</code>	自己写	自己写
输入	手动 <code>`BufferedReader`</code>	平台直接传参(方法参数)
输出	<code>`out.println`</code> + <code>`flush`</code>	<code>`return`</code> 结果

3.2 最简骨架(LeetCode 1 两数之和为例)



```
import java.util.*;

class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> idx = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];
            if (idx.containsKey(need)) {
                return new int[] {idx.get(need), i};
            }
            idx.put(nums[i], i);
        }
        return new int[0]; // 题目保证有解,实际不会走到
    }
}
```

可抄重点:

- class Solution 是固定类名(LeetCode 强制)
- 方法签名严格匹配题目(返回类型 / 方法名 / 参数,一字不差)
- 自定义数据结构(如 ListNode / TreeNode)和 class Solution 放同一个文件即可,LeetCode 编译器认

3.3 自定义 `Comparator` 在 LeetCode 的写法

PriorityQueue 用自定义比较器时,LeetCode 模式有 3 种写法,优先用 lambda(最简洁):

```
import java.util.*;

// 写法 1:lambda(最常用,推荐)
PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) -> b[0] - a[0]);

// 写法 2:Comparator.reverseOrder()(配合 Comparable 时用)
PriorityQueue<Integer> maxHeap2 = new PriorityQueue<>(Comparator.reverseOrder());

// 写法 3:匿名内部类(老写法,可读但啰嗦)
PriorityQueue<int[]> maxHeap3 = new PriorityQueue<>(new Comparator<int[]>() {
    @Override
    public int compare(int[] a, int[] b) {
        return b[0] - a[0];
    }
});
```

3.4 `Arrays.sort` vs `Collections.sort` vs `List.sort` (别混)

方法	适用对象	备注
`Arrays.sort(arr)`	数组(`int[]` / `Integer[]` / `Object[]`)	基本类型用快速排序(快但不稳定);对象用归并(稳定)
`Arrays.sort(arr, fromIndex, toIndex)`	数组 + 范围	半开区间 `[from, to)`,不是闭区间
`Collections.sort(list)`	`List<T>` (如 `ArrayList`)	稳定,Java 8 后建议用 `list.sort` 替代
`list.sort(cmp)`	`List<T>` + `Comparator`	Java 8 新增,比 `Collections.sort` 快(Java 内部实现差异)
`Arrays.sort(arr, cmp)`	对象数组 + `Comparator`	用于 `Integer[]` 等对象数组自定义排序



实战选型:

- 刷 LeetCode, 排序 `int[]` → `Arrays.sort(arr)`
- 排序 `List<Integer>` → `list.sort(Comparator.reverseOrder())` (升序就 `list.sort(null)` 或 `Collections.sort`)
- 不要用 `Collections.sort`, 除非刷老题或题解抄过来的

C++ 对照:

```
#include <algorithm>
#include <vector>

std::vector<int> v = {3, 1, 2};

std::sort(v.begin(), v.end());           // 升序(对应 Collections.sort / list.sort)
std::sort(v.begin(), v.end(), std::greater<int>()); // 降序(对应 list.sort(Comparator.reverseOrder()))
```

四、10 大算法 Java 模板(必背)

说明:本节是本章核心。每个算法给"完整可抄 Java 代码 + C++ 对照 + 思路一句话"。不展开算法原理(那是《算法导论》的事),只给模板。

>

使用建议:先抄 1 遍,抄完才记得住。光看是记不住的,这是刷题模板的金科玉律。

4.1 排序

4.1.1 快速排序(完整 Java 实现)

思路:选基准值(通常 `arr[l]`),双指针从两端向中间,把小于基准的放左边、大于基准的放右边,递归处理左右两段。



```

import java.util.*;

public class Solution {
    public int[] sortArray(int[] arr) {
        quickSort(arr, 0, arr.length - 1);
        return arr;
    }

    private void quickSort(int[] arr, int l, int r) {
        if (l >= r) return;          // 递归出口
        int pivot = arr[l];          // 选第一个为基准
        int i = l, j = r;
        while (i < j) {
            while (i < j && arr[j] >= pivot) j--; // 右边找小于基准的
            while (i < j && arr[i] <= pivot) i++; // 左边找大于基准的
            if (i < j) {
                int tmp = arr[i];
                arr[i] = arr[j];
                arr[j] = tmp;
            }
        }
        // i == j, 基准归位
        arr[l] = arr[i];
        arr[i] = pivot;
        quickSort(arr, l, i - 1);
        quickSort(arr, i + 1, r);
    }
}

```

C++ 对照(标准库一行搞定,这里给手写版):

```

#include <vector>
#include <algorithm>
void quickSort(std::vector<int>& arr, int l, int r) {
    if (l >= r) return;
    int pivot = arr[l];
    int i = l, j = r;
    while (i < j) {
        while (i < j && arr[j] >= pivot) j--;
        while (i < j && arr[i] <= pivot) i++;
        if (i < j) std::swap(arr[i], arr[j]);
    }
    std::swap(arr[l], arr[i]);
    quickSort(arr, l, i - 1);
    quickSort(arr, i + 1, r);
}

```

金句:Java 刷题 90% 场景直接 `Arrays.sort` 就行,快排手写版只在面试要求手写时才用。`Arrays.sort` 底层对基本类型是 Dual-Pivot QuickSort,对对象是 TimSort,都比手写快。

4.1.2 归并排序(完整 Java 实现)

思路:分治(分到只剩 1 个元素自然有序)→ 合并两个有序数组(双指针 + 临时数组)。



```

import java.util.*;

public class Solution {
    public int[] sortArray(int[] arr) {
        if (arr.length <= 1) return arr;
        int[] temp = new int[arr.length]; // 全局临时数组,避免反复 new
        mergeSort(arr, 0, arr.length - 1, temp);
        return arr;
    }

    private void mergeSort(int[] arr, int l, int r, int[] temp) {
        if (l >= r) return;
        int mid = l + (r - l) / 2;
        mergeSort(arr, l, mid, temp);
        mergeSort(arr, mid + 1, r, temp);
        merge(arr, l, mid, r, temp);
    }

    private void merge(int[] arr, int l, int mid, int r, int[] temp) {
        int i = l, j = mid + 1, k = l;
        while (i <= mid && j <= r) {
            if (arr[i] <= arr[j]) {
                temp[k++] = arr[i++];
            } else {
                temp[k++] = arr[j++];
            }
        }
        while (i <= mid) temp[k++] = arr[i++];
        while (j <= r) temp[k++] = arr[j++];
        for (int p = l; p <= r; p++) {
            arr[p] = temp[p];
        }
    }
}

```

C++ 对照:

```

#include <vector>

void mergeSort(std::vector<int>& arr, int l, int r, std::vector<int>& temp) {
    if (l >= r) return;
    int mid = l + (r - l) / 2;
    mergeSort(arr, l, mid, temp);
    mergeSort(arr, mid + 1, r, temp);
    int i = l, j = mid + 1, k = l;
    while (i <= mid && j <= r) {
        if (arr[i] <= arr[j]) temp[k++] = arr[i++];
        else temp[k++] = arr[j++];
    }
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= r) temp[k++] = arr[j++];
    for (int p = l; p <= r; p++) arr[p] = temp[p];
}

```



金句:归并排序是稳定的,在"求逆序对数 / 链表排序"等场景比快排更合适。

4.1.3 堆排序(`PriorityQueue` 间接实现)

Java 没暴露堆内部结构,刷题用 `PriorityQueue` 间接实现堆排:

```
import java.util.*;

public class Solution {
    public int[] sortArray(int[] arr) {
        // 小顶堆:堆顶是最小,逐个 poll 出来就是升序
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        for (int x : arr) minHeap.offer(x);
        int[] res = new int[arr.length];
        for (int i = 0; i < arr.length; i++) {
            res[i] = minHeap.poll();
        }
        return res;
    }
}
```

C++ 对照:

```
#include <queue>
#include <vector>
std::priority_queue<int, std::vector<int>, std::greater<int>>> minHeap;
for (int x : arr) minHeap.push(x);
for (int i = 0; i < arr.size(); i++) {
    arr[i] = minHeap.top(); minHeap.pop();
}
```

金句:`PriorityQueue` 调 `Arrays.sort` 本身也快,实战中"堆排"几乎只用于 Top K 问题(LeetCode 215 数组中的第 K 个最大元素)。

4.2 二分(高频)

4.2.1 标准二分(找一个数)

思路:有序数组找 `target`,返回下标(找不到返回 -1)。左闭右闭写法。




```
import java.util.*;

public class Solution {
    public int search(int[] nums, int target) {
        int l = 0, r = nums.length - 1;    // [l, r] 闭区间
        while (l <= r) {
            int mid = l + (r - l) / 2;    // 防溢出,等价于 (l + r) / 2
            if (nums[mid] == target) return mid;
            else if (nums[mid] < target) l = mid + 1;
            else r = mid - 1;
        }
        return -1;
    }
}
```

C++ 对照:

```
#include <vector>
int search(std::vector<int>& nums, int target) {
    int l = 0, r = nums.size() - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) l = mid + 1;
        else r = mid - 1;
    }
    return -1;
}
```

4.2.2 找左边界(第一个 $\geq$ target 的位置)

思路:lower_bound。找到第一个大于等于 target 的下标,常用于"插入位置"。

```
import java.util.*;

public class Solution {
    public int lowerBound(int[] nums, int target) {
        int l = 0, r = nums.length;    // [l, r) 半开区间
        while (l < r) {
            int mid = l + (r - l) / 2;
            if (nums[mid] < target) {
                l = mid + 1;
            } else {
                r = mid;    // nums[mid] >= target, mid 可能是答案
            }
        }
        return l;    // l == r, 第一个 >= target 的位置
    }
}
```

C++ 对照:



```
#include <algorithm>
#include <vector>

int lowerBound(std::vector<int>& nums, int target) {
    return std::lower_bound(nums.begin(), nums.end(), target) - nums.begin();
}
```

4.2.3 找右边界(最后一个 $\leq$ target 的位置)

思路:upper_bound(target + 1) 或者改写为"找第一个 $>$ target 再减 1"。这里给直接写法。

```
import java.util.*;

public class Solution {
    public int upperBound(int[] nums, int target) {
        int l = 0, r = nums.length;    // [l, r) 半开区间
        while (l < r) {
            int mid = l + (r - l) / 2;
            if (nums[mid] <= target) {
                l = mid + 1;    // mid 还可以往右扩
            } else {
                r = mid;
            }
        }
        return l - 1;    // 最后一个 <= target 的位置
    }
}
```

C++ 对照:

```
#include <algorithm>
#include <vector>

int upperBound(std::vector<int>& nums, int target) {
    return std::upper_bound(nums.begin(), nums.end(), target) - nums.begin() - 1;
}
```

4.2.4 二分模板选择

模板	区间	退出条件	适用
标准二分(找一个数)	<code>[l, r]</code> 闭	<code>l > r</code>	找等于 target 的下标
找左边界	<code>[l, r)</code> 半开	<code>l == r</code>	找第一个 $\geq$ target
找右边界	<code>[l, r]</code> 半开	<code>l == r</code>	找最后一个 $\leq$ target

金句:"左闭右闭 vs 左闭右开"两种模板,二选一坚持到底。混用是 90% 二分 bug 的来源。

4.3 双指针(高频)

4.3.1 对撞指针(LeetCode 167 两数之和 II)

思路:有序数组,左右指针从两端往中间,根据和与 target 的关系移动。



```
import java.util.*;

public class Solution {
    public int[] twoSum(int[] numbers, int target) {
        int l = 0, r = numbers.length - 1;
        while (l < r) {
            int sum = numbers[l] + numbers[r];
            if (sum == target) {
                return new int[] {l + 1, r + 1}; // 题目要求 1-indexed
            } else if (sum < target) {
                l++;
            } else {
                r--;
            }
        }
        return new int[] {-1, -1};
    }
}
```

C++ 对照:

```
#include <vector>

std::vector<int> twoSum(std::vector<int>& numbers, int target) {
    int l = 0, r = numbers.size() - 1;
    while (l < r) {
        int sum = numbers[l] + numbers[r];
        if (sum == target) return {l + 1, r + 1};
        else if (sum < target) l++;
        else r--;
    }
    return {-1, -1};
}
```

4.3.2 快慢指针(LeetCode 141 环形链表)

思路:慢指针走 1 步,快指针走 2 步;相遇则有环。



```

import java.util.*;

class ListNode {
    int val;
    ListNode next;
    ListNode(int x) { val = x; next = null; }
}

public class Solution {
    public boolean hasCycle(ListNode head) {
        if (head == null || head.next == null) return false;
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;    // 慢走 1 步
            fast = fast.next.next; // 快走 2 步
            if (slow == fast) return true;
        }
        return false;
    }
}

```

C++ 对照:

```

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

bool hasCycle(ListNode* head) {
    if (!head || !head->next) return false;
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}

```

4.3.3 滑动窗口(LeetCode 3 无重复字符的最长子串)

思路:维护一个 $[l, r]$ 窗口,不断扩大右边界,出现重复就缩小左边界,记录最大窗口。



```

import java.util.*;

public class Solution {
    public int lengthOfLongestSubstring(String s) {
        Map<Character, Integer> window = new HashMap<>(); // 字符 → 最新下标
        int l = 0, ans = 0;
        for (int r = 0; r < s.length(); r++) {
            char c = s.charAt(r);
            if (window.containsKey(c)) {
                // 关键:左边界只往右移,不回头
                l = Math.max(l, window.get(c) + 1);
            }
            window.put(c, r);
            ans = Math.max(ans, r - l + 1);
        }
        return ans;
    }
}

```

C++ 对照:

```

#include <string>
#include <unordered_map>
#include <algorithm>
int lengthOfLongestSubstring(std::string s) {
    std::unordered_map<char, int> window;
    int l = 0, ans = 0;
    for (int r = 0; r < (int)s.size(); r++) {
        if (window.count(s[r])) {
            l = std::max(l, window[s[r]] + 1);
        }
        window[s[r]] = r;
        ans = std::max(ans, r - l + 1);
    }
    return ans;
}

```

金句:滑动窗口 = 双指针 + 哈希表。看到"最长/最短/恰好 K 个"子串/子数组,先想滑动窗口。

4.4 BFS / DFS (高频)

4.4.1 BFS 模板 (LeetCode 102 二叉树的层序遍历)

思路:ArrayDeque 当队列,层序遍历每层节点,先入先出。



```

import java.util.*;

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int v) { val = v; }
}

public class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        if (root == null) return res;
        ArrayDeque<TreeNode> q = new ArrayDeque<>();
        q.offer(root);
        while (!q.isEmpty()) {
            int sz = q.size();          // 当前层节点数,固定
            List<Integer> level = new ArrayList<>(sz);
            for (int i = 0; i < sz; i++) {
                TreeNode node = q.poll();
                level.add(node.val);
                if (node.left != null) q.offer(node.left);
                if (node.right != null) q.offer(node.right);
            }
            res.add(level);
        }
        return res;
    }
}

```

C++ 对照:

```

#include <vector>
#include <queue>
std::vector<std::vector<int>> levelOrder(TreeNode* root) {
    std::vector<std::vector<int>> res;
    if (!root) return res;
    std::queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        std::vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}

```

金句:BFS 求最短路(无权图):第一次到达即为最短。迷宫、单词接龙、腐烂橘子(LeetCode 994)都靠这招



。

4.4.2 DFS 模板:递归(LeetCode 104 二叉树最大深度)

思路:递归,左右子树深度 +1 取 max。

```
import java.util.*;

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int v) { val = v; }
}

public class Solution {
    public int maxDepth(TreeNode root) {
        if (root == null) return 0;
        int left = maxDepth(root.left);
        int right = maxDepth(root.right);
        return Math.max(left, right) + 1;
    }
}
```

C++ 对照:

```
int maxDepth(TreeNode* root) {
    if (!root) return 0;
    int left = maxDepth(root->left);
    int right = maxDepth(root->right);
    return std::max(left, right) + 1;
}
```

4.4.3 DFS 模板:栈(迭代写法,防栈溢出)

思路:用 ArrayDeque 模拟栈,显式管理"节点 + 已访问状态"。适合深度很大的树(LeetCode 默认栈深 ~1 万,递归深一点就 StackOverflowError)。



```

import java.util.*;

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int v) { val = v; }
}

public class Solution {
    public List<Integer> preorderTraversal(TreeNode root) {
        List<Integer> res = new ArrayList<>();
        if (root == null) return res;
        ArrayDeque<TreeNode> stack = new ArrayDeque<>();
        stack.push(root);
        while (!stack.isEmpty()) {
            TreeNode node = stack.pop();
            res.add(node.val);
            // 先压右再压左.弹出来才是根→左→右
            if (node.right != null) stack.push(node.right);
            if (node.left != null) stack.push(node.left);
        }
        return res;
    }
}

```

C++ 对照:

```

#include <vector>
#include <stack>
std::vector<int> preorderTraversal(TreeNode* root) {
    std::vector<int> res;
    if (!root) return res;
    std::stack<TreeNode*> st;
    st.push(root);
    while (!st.empty()) {
        TreeNode* node = st.top(); st.pop();
        res.push_back(node->val);
        if (node->right) st.push(node->right);
        if (node->left) st.push(node->left);
    }
    return res;
}

```

金句:递归易写,但深树必爆栈。刷题默认写递归,真爆栈再改迭代。

4.5 DP(高频)

金句:"DP 三步:定义状态 → 写转移方程 → 初始化边界"。第三步最容易漏。

4.5.1 一维 DP(LeetCode 70 爬楼梯)

思路: $dp[i]$ = 爬到第 i 阶的方法数。 $dp[i] = dp[i-1] + dp[i-2]$ 。




```
import java.util.*;

public class Solution {
    public int climbStairs(int n) {
        if (n <= 2) return n;
        int prev2 = 1, prev1 = 2;    // 滚动数组,空间 O(1)
        for (int i = 3; i <= n; i++) {
            int cur = prev1 + prev2;
            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;
    }
}
```

C++ 对照:

```
int climbStairs(int n) {
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++) {
        int cur = prev1 + prev2;
        prev2 = prev1;
        prev1 = cur;
    }
    return prev1;
}
```

4.5.2 二维 DP(LeetCode 62 不同路径)

思路: $dp[i][j]$ = 到 (i, j) 的路径数。 $dp[i][j] = dp[i-1][j] + dp[i][j-1]$ 。

```
import java.util.*;

public class Solution {
    public int uniquePaths(int m, int n) {
        int[][] dp = new int[m][n];
        for (int i = 0; i < m; i++) dp[i][0] = 1;    // 第一列
        for (int j = 0; j < n; j++) dp[0][j] = 1;    // 第一行
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = dp[i-1][j] + dp[i][j-1];
            }
        }
        return dp[m-1][n-1];
    }
}
```

C++ 对照:



```
#include <vector>

int uniquePaths(int m, int n) {
    std::vector<std::vector<int>> dp(m, std::vector<int>(n, 1));
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
    return dp[m - 1][n - 1];
}
```

4.5.3 0-1 背包(LeetCode 416 分割等和子集)

思路:0-1 背包 = 每个物品选或不选(一次)。dp[j] = 是否能凑出和 j。逆序遍历避免重复选。

```
import java.util.*;

public class Solution {
    public boolean canPartition(int[] nums) {
        int sum = 0;
        for (int x : nums) sum += x;
        if (sum % 2 != 0) return false;    // 奇数不可能平分
        int target = sum / 2;
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;    // 边界:和为 0 一定可达
        for (int num : nums) {
            for (int j = target; j >= num; j--) {    // ★ 逆序!0-1 背包的关键
                dp[j] = dp[j] || dp[j - num];
            }
        }
        return dp[target];
    }
}
```

C++ 对照:

```
#include <vector>

bool canPartition(std::vector<int>& nums) {
    int sum = 0;
    for (int x : nums) sum += x;
    if (sum % 2) return false;
    int target = sum / 2;
    std::vector<bool> dp(target + 1, false);
    dp[0] = true;
    for (int num : nums) {
        for (int j = target; j >= num; j--) {
            dp[j] = dp[j] || dp[j - num];
        }
    }
    return dp[target];
}
```

4.5.4 完全背包(LeetCode 322 零钱兑换)

思路:完全背包 = 每个物品可以选无限次。dp[j] = 凑出金额 j 的最少硬币数。正序遍历。



```
import java.util.*;

public class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1); // 初始化为"无穷大"
        dp[0] = 0; // 边界:金额 0 需要 0 个硬币
        for (int coin : coins) {
            for (int j = coin; j <= amount; j++) { // ★ 正序!完全背包的关键
                dp[j] = Math.min(dp[j], dp[j - coin] + 1);
            }
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }
}
```

C++ 对照:

```
#include <vector>
#include <algorithm>

int coinChange(std::vector<int>& coins, int amount) {
    std::vector<int> dp(amount + 1, amount + 1);
    dp[0] = 0;
    for (int coin : coins) {
        for (int j = coin; j <= amount; j++) {
            dp[j] = std::min(dp[j], dp[j - coin] + 1);
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

金句:0-1 背包逆序,完全背包正序。记反就是 80% DP 错的来源。

4.6 并查集(高频)

4.6.1 模板(完整 Java 实现,带路径压缩 + 按秩合并)

思路:把"互相连通"的元素放进同一集合。find 找根 + 路径压缩;union 按秩合并。



```

import java.util.*;

public class Solution {
    private int[] parent; // parent[i] = i 的父节点,根的 parent 是自己
    private int[] rank;   // 秩(树高度上界)

    public Solution(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i; // 初始时各自为根
            rank[i] = 1;
        }
    }

    // 找 x 的根,带路径压缩
    public int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]); // 递归压缩
        }
        return parent[x];
    }

    // 合并 x 和 y 所在的集合,带按秩合并
    public void union(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return;
        if (rank[rx] < rank[ry]) {
            parent[rx] = ry;
        } else if (rank[rx] > rank[ry]) {
            parent[ry] = rx;
        } else {
            parent[ry] = rx;
            rank[rx]++;
        }
    }

    // 判断 x 和 y 是否在同一个集合
    public boolean connected(int x, int y) {
        return find(x) == find(y);
    }
}

```

C++ 对照:



```

#include <vector>
class UnionFind {
    std::vector<int> parent, rank;
public:
    UnionFind(int n) : parent(n), rank(n, 1) {
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    void unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return;
        if (rank[rx] < rank[ry]) parent[rx] = ry;
        else if (rank[rx] > rank[ry]) parent[ry] = rx;
        else { parent[ry] = rx; rank[rx]++; }
    }
    bool connected(int x, int y) { return find(x) == find(y); }
};

```

4.6.2 LeetCode 200 岛屿数量(并查集解法)

思路:遍历网格,1 周围(上下左右)是 1 就 union。最后统计有几个根。



```

import java.util.*;

public class Solution {
    public int numIslands(char[][] grid) {
        if (grid == null || grid.length == 0) return 0;
        int m = grid.length, n = grid[0].length;
        int[] parent = new int[m * n];
        int[] rank = new int[m * n];
        for (int i = 0; i < m * n; i++) {
            parent[i] = i;
            rank[i] = 1;
        }
        int count = 0;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] != '1') continue;
                count++;
                int idx = i * n + j;
                if (i + 1 < m && grid[i + 1][j] == '1') union(parent, rank, idx, idx + n);
                if (j + 1 < n && grid[i][j + 1] == '1') union(parent, rank, idx, idx + 1);
            }
        }
        // 统计根的个数
        int roots = 0;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == '1' && parent[i * n + j] == i * n + j) roots++;
            }
        }
        return roots;
    }

    private int find(int[] parent, int x) {
        if (parent[x] != x) parent[x] = find(parent, parent[x]);
        return parent[x];
    }

    private void union(int[] parent, int[] rank, int x, int y) {
        int rx = find(parent, x), ry = find(parent, y);
        if (rx == ry) return;
        if (rank[rx] < rank[ry]) parent[rx] = ry;
        else if (rank[rx] > rank[ry]) parent[ry] = rx;
        else { parent[ry] = rx; rank[rx]++; }
        // 注意:count 已在主循环里减
    }
}

```

实战提醒:LeetCode 200 多数题解用 DFS / BFS,实现更短;并查集解法适合"朋友圈"(LeetCode 547)等场景。

C++ 对照(DFS 版,更常见):



```

#include <vector>

void dfs(std::vector<std::vector<char>>& grid, int i, int j) {
    int m = grid.size(), n = grid[0].size();
    if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] != '1') return;
    grid[i][j] = '0';
    dfs(grid, i + 1, j); dfs(grid, i - 1, j);
    dfs(grid, i, j + 1); dfs(grid, i, j - 1);
}

int numIslands(std::vector<std::vector<char>>& grid) {
    int count = 0;
    for (int i = 0; i < (int)grid.size(); i++)
        for (int j = 0; j < (int)grid[0].size(); j++)
            if (grid[i][j] == '1') { count++; dfs(grid, i, j); }
    return count;
}

```

4.7 单调栈 / 单调队列

4.7.1 单调栈(LeetCode 739 每日温度)

思路:维护一个下标栈,栈内下标对应的温度单调递减。遇到更高温度时,栈顶出栈并记录答案。

```

import java.util.*;

public class Solution {
    public int[] dailyTemperatures(int[] T) {
        int n = T.length;
        int[] ans = new int[n];
        ArrayDeque<Integer> stack = new ArrayDeque<>(); // ★ 存下标,不是值
        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && T[stack.peek()] < T[i]) {
                int prev = stack.pop();
                ans[prev] = i - prev;          // 距离
            }
            stack.push(i);
        }
        return ans;
    }
}

```

C++ 对照:



```

#include <vector>
#include <stack>
std::vector<int> dailyTemperatures(std::vector<int>& T) {
    int n = T.size();
    std::vector<int> ans(n, 0);
    std::stack<int> st;          // 存下标
    for (int i = 0; i < n; i++) {
        while (!st.empty() && T[st.top()] < T[i]) {
            int prev = st.top(); st.pop();
            ans[prev] = i - prev;
        }
        st.push(i);
    }
    return ans;
}

```

4.7.2 单调队列(LeetCode 239 滑动窗口最大值)

思路: ArrayDeque 存下标, 维护单调递减。队首永远是当前窗口最大值。

```

import java.util.*;

public class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int n = nums.length;
        int[] ans = new int[n - k + 1];
        ArrayDeque<Integer> deque = new ArrayDeque<>(); // ★ 存下标
        for (int i = 0; i < n; i++) {
            // 1. 移除窗口外的下标
            while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) {
                deque.pollFirst();
            }
            // 2. 维护单调递减(从队尾删小的)
            while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) {
                deque.pollLast();
            }
            deque.offerLast(i);
            // 3. 窗口形成后记录答案
            if (i >= k - 1) {
                ans[i - k + 1] = nums[deque.peekFirst()];
            }
        }
        return ans;
    }
}

```

C++ 对照:




```

#include <vector>
#include <deque>
std::vector<int> maxSlidingWindow(std::vector<int>& nums, int k) {
    int n = nums.size();
    std::vector<int> ans(n - k + 1);
    std::deque<int> dq;           // 存下标
    for (int i = 0; i < n; i++) {
        while (!dq.empty() && dq.front() < i - k + 1) dq.pop_front();
        while (!dq.empty() && nums[dq.back()] < nums[i]) dq.pop_back();
        dq.push_back(i);
        if (i >= k - 1) ans[i - k + 1] = nums[dq.front()];
    }
    return ans;
}

```

金句:"单调栈存下标,单调队列存下标,都是`ArrayDeque`"。peekFirst 是队首/栈顶,peekLast 是队尾,offerLast / pollLast 是尾入尾出。

4.8 前缀和 / 差分

4.8.1 一维前缀和(LeetCode 303 区域和检索)

思路:pre[i] = 前 i 个元素之和(pre[0] = 0)。区间和 [l, r] = pre[r + 1] - pre[l]。

```

import java.util.*;

public class Solution {
    private int[] pre; // pre[0] = 0,pre[i+1] = nums[0..i] 的和

    public Solution(int[] nums) { // 构造时算前缀和
        int n = nums.length;
        pre = new int[n + 1];
        for (int i = 0; i < n; i++) {
            pre[i + 1] = pre[i] + nums[i];
        }
    }

    public int sumRange(int left, int right) {
        return pre[right + 1] - pre[left];
    }
}

```

C++ 对照:



```

#include <vector>
class NumArray {
    std::vector<int> pre;
public:
    NumArray(std::vector<int>& nums) {
        int n = nums.size();
        pre.resize(n + 1, 0);
        for (int i = 0; i < n; i++) pre[i + 1] = pre[i] + nums[i];
    }
    int sumRange(int left, int right) {
        return pre[right + 1] - pre[left];
    }
};

```

4.8.2 二维前缀和(LeetCode 304 二维区域和检索)

思路: $pre[i][j]$ = 左上角 (0, 0) 到 (i - 1, j - 1) 的矩阵和。 $pre[0][*]$ 和 $pre[*][0]$ 全是 0, 方便边界。

```

import java.util.*;

public class Solution {
    private int[][] pre;

    public Solution(int[][] matrix) {
        if (matrix == null || matrix.length == 0) return;
        int m = matrix.length, n = matrix[0].length;
        pre = new int[m + 1][n + 1];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                pre[i + 1][j + 1] = pre[i + 1][j] + pre[i][j + 1] - pre[i][j] + matrix[i][j];
            }
        }
    }

    public int sumRegion(int row1, int col1, int row2, int col2) {
        return pre[row2 + 1][col2 + 1] - pre[row1][col2 + 1]
            - pre[row2 + 1][col1] + pre[row1][col1];
    }
}

```

C++ 对照:



```

#include <vector>
class NumMatrix {
    std::vector<std::vector<int>>> pre;
public:
    NumMatrix(std::vector<std::vector<int>>& matrix) {
        if (matrix.empty()) return;
        int m = matrix.size(), n = matrix[0].size();
        pre.assign(m + 1, std::vector<int>(n + 1, 0));
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                pre[i + 1][j + 1] = pre[i + 1][j] + pre[i][j + 1] - pre[i][j] + matrix[i][j];
    }
    int sumRegion(int r1, int c1, int r2, int c2) {
        return pre[r2 + 1][c2 + 1] - pre[r1][c2 + 1] - pre[r2 + 1][c1] + pre[r1][c1];
    }
};

```

4.8.3 差分数组(LeetCode 1094 拼车)

思路: $\text{diff}[i] = \text{arr}[i] - \text{arr}[i - 1]$ 。区间 $[l, r]$ 统一加 val , 只需 $\text{diff}[l] += \text{val}$; $\text{diff}[r + 1] -= \text{val}$ 。最后前缀和还原。

```

import java.util.*;

public class Solution {
    public boolean carPooling(int[][] trips, int capacity) {
        // 站点 0~1000, 差分数组
        int[] diff = new int[1001];
        for (int[] trip : trips) {
            int num = trip[0], from = trip[1], to = trip[2];
            diff[from] += num;
            diff[to] -= num;    // 在 to 站下车, 所以 [from, to) 加 num
        }
        // 前缀和还原 + 判断是否超员
        int cur = 0;
        for (int i = 0; i < 1001; i++) {
            cur += diff[i];
            if (cur > capacity) return false;
        }
        return true;
    }
}

```

C++ 对照:



```

#include <vector>
bool carPooling(std::vector<std::vector<int>>& trips, int capacity) {
    std::vector<int> diff(1001, 0);
    for (auto& trip : trips) {
        diff[trip[1]] += trip[0];
        diff[trip[2]] -= trip[0];
    }
    int cur = 0;
    for (int i = 0; i < 1001; i++) {
        cur += diff[i];
        if (cur > capacity) return false;
    }
    return true;
}

```

金句:前缀和解决"静态区间求和",差分解决"区间统一修改"。两者互为逆运算。

4.9 回溯(新增,刷题必学)

4.9.1 三件套与模板

三件套:

- result:结果集(所有合法解)
- path:当前路径(正在构造的一个解)
- backtrack(选择列表):递归函数

思路:选一个 → 递归 → 撤销选择(回溯)。

```

import java.util.*;

public class Solution {
    public List<List<Integer>> result = new ArrayList<>(); // 所有解
    public LinkedList<Integer> path = new LinkedList<>(); // 当前路径(用 LinkedList 是为了 O(1) 头尾操作)

    public void backtrack(int[] nums) {
        if (path.size() == nums.length) { // ★ 终止条件
            result.add(new ArrayList<>(path)); // 注意:拷贝一份!
            return;
        }
        for (int i = 0; i < nums.length; i++) {
            if (path.contains(nums[i])) continue; // 剪枝:已选过就跳过
            path.add(nums[i]); // 1. 做选择
            backtrack(nums); // 2. 递归
            path.removeLast(); // 3. 撤销选择
        }
    }
}

```

关键:"撤销选择"是回溯的核心。add 完递归完,必须 removeLast / remove 还原,否则下一轮路径会错。

4.9.2 LeetCode 46 全排列(无重复)



```

import java.util.*;

public class Solution {
    private List<List<Integer>> result = new ArrayList<>();
    private LinkedList<Integer> path = new LinkedList<>();
    private boolean[] used;

    public List<List<Integer>> permute(int[] nums) {
        used = new boolean[nums.length];
        backtrack(nums);
        return result;
    }

    private void backtrack(int[] nums) {
        if (path.size() == nums.length) {
            result.add(new ArrayList<>(path));
            return;
        }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;           // 用 used 数组判断,比 contains 快
            used[i] = true;
            path.add(nums[i]);
            backtrack(nums);
            path.removeLast();
            used[i] = false;                 // 撤销
        }
    }
}

```

C++ 对照:



```

#include <vector>
class Solution {
    std::vector<std::vector<int>>> result;
    std::vector<int> path;
    std::vector<bool> used;
    void backtrack(std::vector<int>& nums) {
        if (path.size() == nums.size()) {
            result.push_back(path);
            return;
        }
        for (int i = 0; i < (int)nums.size(); i++) {
            if (used[i]) continue;
            used[i] = true;
            path.push_back(nums[i]);
            backtrack(nums);
            path.pop_back();
            used[i] = false;
        }
    }
public:
    std::vector<std::vector<int>>> permute(std::vector<int>& nums) {
        used.assign(nums.size(), false);
        backtrack(nums);
        return result;
    }
};

```

4.9.3 LeetCode 78 子集

思路:用 `start` 索引去重(避免 [1, 2] 和 [2, 1] 重复)。

```

import java.util.*;

public class Solution {
    private List<List<Integer>> result = new ArrayList<>();
    private LinkedList<Integer> path = new LinkedList<>();

    public List<List<Integer>> subsets(int[] nums) {
        backtrack(nums, 0);
        return result;
    }

    private void backtrack(int[] nums, int start) {
        result.add(new ArrayList<>(path)); // ★ 每个 path 都是一个子集
        for (int i = start; i < nums.length; i++) {
            path.add(nums[i]);
            backtrack(nums, i + 1); // i + 1,避免回头选
            path.removeLast();
        }
    }
}

```

C++ 对照:



```
#include <vector>
class Solution {
    std::vector<std::vector<int>>> result;
    std::vector<int> path;
    void backtrack(std::vector<int>& nums, int start) {
        result.push_back(path);
        for (int i = start; i < (int)nums.size(); i++) {
            path.push_back(nums[i]);
            backtrack(nums, i + 1);
            path.pop_back();
        }
    }
public:
    std::vector<std::vector<int>>> subsets(std::vector<int>& nums) {
        backtrack(nums, 0);
        return result;
    }
};
```

4.9.4 LeetCode 47 全排列 II(有重复,需剪枝)

思路:先排序,让相同元素相邻;再剪枝 —— 同层若前一个相同元素还没用,当前就不能用。



```

import java.util.*;

public class Solution {
    private List<List<Integer>> result = new ArrayList<>();
    private LinkedList<Integer> path = new LinkedList<>();
    private boolean[] used;

    public List<List<Integer>> permuteUnique(int[] nums) {
        Arrays.sort(nums);           // ★ 先排序是关键
        used = new boolean[nums.length];
        backtrack(nums);
        return result;
    }

    private void backtrack(int[] nums) {
        if (path.size() == nums.length) {
            result.add(new ArrayList<>(path));
            return;
        }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            // ★ 关键剪枝:同层前一个相同元素未用,跳过
            if (i > 0 && nums[i] == nums[i - 1] && !used[i - 1]) continue;
            used[i] = true;
            path.add(nums[i]);
            backtrack(nums);
            path.removeLast();
            used[i] = false;
        }
    }
}

```

C++ 对照:




```

#include <vector>
#include <algorithm>
class Solution {
    std::vector<std::vector<int>>> result;
    std::vector<int> path;
    std::vector<bool> used;
    void backtrack(std::vector<int>& nums) {
        if (path.size() == nums.size()) {
            result.push_back(path);
            return;
        }
        for (int i = 0; i < (int)nums.size(); i++) {
            if (used[i]) continue;
            if (i > 0 && nums[i] == nums[i - 1] && !used[i - 1]) continue;
            used[i] = true;
            path.push_back(nums[i]);
            backtrack(nums);
            path.pop_back();
            used[i] = false;
        }
    }
public:
    std::vector<std::vector<int>>> permuteUnique(std::vector<int>& nums) {
        std::sort(nums.begin(), nums.end());
        used.assign(nums.size(), false);
        backtrack(nums);
        return result;
    }
};

```

4.9.5 剪枝技巧小结

剪枝手段	适用	例子
`used[]` 数组	排列问题(全排列)	LeetCode 46 / 47
`start` 索引	子集 / 组合(避免回头)	LeetCode 78 / 77
排序后剪枝	元素有重复	LeetCode 47 / 90
路径判断(`path.size() == k`)	组合数固定大小	LeetCode 77 组合

金句:回溯 = DFS + 撤销选择。看到"所有可能 / 列出所有" → 想回溯。

4.10 贪心(新增,刷题必学)

4.10.1 核心理想

核心理想:局部最优 → 全局最优。没有固定模板,关键是排序后看怎么选。

贪心的难点不在"怎么写",而在"怎么证明贪心策略是对的"。本节给策略 + 例题,完整证明不做(数学归纳法 / 交换论证,那是算法课的事)。

4.10.2 LeetCode 55 跳跃游戏(覆盖范围贪心)

思路:维护一个"最远可达下标" maxReach,遍历时更新;若当前下标 > maxReach,则到不了。



```
import java.util.*;

public class Solution {
    public boolean canJump(int[] nums) {
        int maxReach = 0;           // 最远可达下标
        for (int i = 0; i < nums.length; i++) {
            if (i > maxReach) return false;    // 当前位置不可达
            maxReach = Math.max(maxReach, i + nums[i]); // 更新最远
        }
        return true;
    }
}
```

C++ 对照:

```
#include <vector>
#include <algorithm>
bool canJump(std::vector<int>& nums) {
    int maxReach = 0;
    for (int i = 0; i < (int)nums.size(); i++) {
        if (i > maxReach) return false;
        maxReach = std::max(maxReach, i + nums[i]);
    }
    return true;
}
```

4.10.3 LeetCode 435 无重叠区间(排序 + 贪心)

思路:按区间结束时间升序排;每次选结束最早且不与上一选中区间重叠的。

```
import java.util.*;

public class Solution {
    public int eraseOverlapIntervals(int[][] intervals) {
        if (intervals.length == 0) return 0;
        Arrays.sort(intervals, (a, b) -> a[1] - b[1]); // ★ 按结束时间升序
        int count = 1;           // 至少能选 1 个
        int end = intervals[0][1];
        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] >= end) {        // 不重叠
                count++;
                end = intervals[i][1];
            }
        }
        return intervals.length - count;        // 总数 - 最多保留 = 最少删除
    }
}
```

C++ 对照:



```

#include <vector>
#include <algorithm>
int eraseOverlapIntervals(std::vector<std::vector<int>>& intervals) {
    if (intervals.empty()) return 0;
    std::sort(intervals.begin(), intervals.end(),
        [](auto& a, auto& b) { return a[1] < b[1]; });
    int count = 1, end = intervals[0][1];
    for (int i = 1; i < (int)intervals.size(); i++) {
        if (intervals[i][0] >= end) { count++; end = intervals[i][1]; }
    }
    return (int)intervals.size() - count;
}

```

4.10.4 LeetCode 455 分发饼干(排序 + 贪心)

思路:胃口和饼干都升序排,小饼干先满足小胃口(最不贪心的方案)。

```

import java.util.*;

public class Solution {
    public int findContentChildren(int[] g, int[] s) {
        Arrays.sort(g);           // 孩子胃口升序
        Arrays.sort(s);           // 饼干尺寸升序
        int i = 0, j = 0;         // i 指向孩子,j 指向饼干
        while (i < g.length && j < s.length) {
            if (s[j] >= g[i]) {    // 这块饼干能喂这个孩子
                i++;              // 孩子 +1
            }
            j++;                  // 不管能不能喂,这块饼干用掉
        }
        return i;                 // 喂饱了几个孩子
    }
}

```

C++ 对照:

```

#include <vector>
#include <algorithm>
int findContentChildren(std::vector<int>& g, std::vector<int>& s) {
    std::sort(g.begin(), g.end());
    std::sort(s.begin(), s.end());
    int i = 0, j = 0;
    while (i < (int)g.size() && j < (int)s.size()) {
        if (s[j] >= g[i]) i++;
        j++;
    }
    return i;
}

```

金句:"贪心没固定模板,关键是排序后看怎么选"。问题可能多种贪心策略,选能证明正确的那种。



五、常用技巧(新增)

5.1 HashMap + 前缀和(LeetCode 560 和为 K 的子数组)

思路:边遍历边算前缀和,用 HashMap 存"前缀和 → 出现次数"。当前前缀和 - 之前某前缀和 = k,说明中间子数组和 = k。

```
import java.util.*;

public class Solution {
    public int subarraySum(int[] nums, int k) {
        Map<Integer, Integer> cnt = new HashMap<>();
        cnt.put(0, 1); // ★ 边界:前缀和 0 出现 1 次(空数组)
        int pre = 0, ans = 0;
        for (int x : nums) {
            pre += x;
            if (cnt.containsKey(pre - k)) { // 找之前的前缀和 = pre - k
                ans += cnt.get(pre - k);
            }
            cnt.put(pre, cnt.getDefault(pre, 0) + 1); // 更新当前前缀和的计数
        }
        return ans;
    }
}
```

C++ 对照:

```
#include <vector>
#include <unordered_map>
int subarraySum(std::vector<int>& nums, int k) {
    std::unordered_map<int, int> cnt;
    cnt[0] = 1; // 边界
    int pre = 0, ans = 0;
    for (int x : nums) {
        pre += x;
        if (cnt.count(pre - k)) ans += cnt[pre - k];
        cnt[pre]++; // 等价于 cnt.getDefault + 1
    }
    return ans;
}
```

金句:"HashMap + 前缀和 = 子数组和等于 K" 的工业标准解法。配合 04 的 `getOrDefault` / `containsKey` 用。`cnt.put(0, 1)` 是最容易漏的边界。

5.2 位运算技巧

5.2.1 `n & (n - 1)`: 去掉最低位的 1

用途:判断 1 的个数 / 判断是否是 2 的幂。



```
import java.util.*;

public class Solution {
    // LeetCode 191 位 1 的个数
    public int hammingWeight(int n) {
        int count = 0;
        while (n != 0) {
            n = n & (n - 1); // ★ 每次去掉最低位的 1
            count++;
        }
        return count;
    }

    // LeetCode 231 2 的幂
    public boolean isPowerOfTwo(int n) {
        return n > 0 && (n & (n - 1)) == 0; // 2 的幂只有一个 1, 去掉后是 0
    }
}
```

C++ 对照:

```
int hammingWeight(int n) {
    int count = 0;
    while (n) { n = n & (n - 1); count++; }
    return count;
}

bool isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

5.2.2 `n & (-n)`: 提取最低位的 1

用途: Lowbit, 获取最低位的 1。常用于树状数组。

```
import java.util.*;

public class Solution {
    // LeetCode 201 数字范围按位与
    public int rangeBitwiseAnd(int left, int right) {
        int shift = 0;
        while (left < right) { // 找公共前缀
            left >>= 1;
            right >>= 1;
            shift++;
        }
        return left << shift;
    }
}
```

5.2.3 异或找唯一 (LeetCode 136 只出现一次的数字)

思路: 异或满足交换律 / 自反性 ($a \oplus a = 0$) / 与 0 异或等于自身。所有数异或 = 出现一次的数。



```
import java.util.*;

public class Solution {
    public int singleNumber(int[] nums) {
        int ans = 0;
        for (int x : nums) {
            ans ^= x;          // 异或三性质:交换律 / 自反 / 与 0 得自身
        }
        return ans;
    }
}
```

C++ 对照:

```
int singleNumber(std::vector<int>& nums) {
    int ans = 0;
    for (int x : nums) ans ^= x;
    return ans;
}
```

5.2.4 异或找缺失(LeetCode 268 缺失数字)

思路:把 $0 \sim n$ 跟 `nums` 一起异或,缺失的那个会剩下。

```
import java.util.*;

public class Solution {
    public int missingNumber(int[] nums) {
        int n = nums.length;
        int ans = 0;
        for (int i = 0; i <= n; i++) ans ^= i;    // 异或  $0..n$ 
        for (int x : nums) ans ^= x;           // 异或数组
        return ans;                            // 剩下的就是缺失的
    }
}
```

C++ 对照:

```
int missingNumber(std::vector<int>& nums) {
    int n = nums.size(), ans = 0;
    for (int i = 0; i <= n; i++) ans ^= i;
    for (int x : nums) ans ^= x;
    return ans;
}
```

金句:"异或三性质:交换律 / 自反性 / 与 0 异或等于自身"。刷"出现一次的数字"系列题(136 / 137 / 260) 必用。

5.3 Trie(进阶,速成可不学)

一句话说明:Trie(前缀树) 用于"字符串前缀匹配"类问题,如 LeetCode 208 实现 Trie、LeetCode 212 单词搜



索 II。

速成阶段:不展开,先记一句口诀——"每个节点一个 26 长度的数组,跑前缀就在树上走"。

何时学:刷到 LeetCode 208 / 212 卡壳时再回来看,届时 06 之后会有补充。

```
// 极简 Trie 节点定义(LeetCode 208 标准,了解即可)
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEnd = false;
}
```

金句:Trie 是面试高频、LeetCode 中频,速成阶段跳过不亏。

六、模板选型速查(忘记选哪个?翻这里)

题目特征	选这个
找某个数 / 找插入位置 / 找边界	二分(4.2)
数组中两数 / 三数之和(有序)	对撞指针(4.3.1)
链表环 / 链表中点	快慢指针(4.3.2)
最长 / 最短 / 恰好 K 个子串或子数组	滑动窗口(4.3.3)
二叉树层序 / 最短路径	BFS(4.4.1)
二叉树深度 / 全路径 / 排列类	DFS 递归(4.4.2)
深度特别大的树	DFS 栈迭代(4.4.3)
爬楼梯 / 路径数 / 最长子序列	DP(4.5)
0-1 背包(每件选一次)	0-1 背包,逆序(4.5.3)
完全背包(每件选无限次)	完全背包,正序(4.5.4)
朋友圈 / 岛屿连通 / 元素分组	并查集(4.6)
下一个更大 / 更小元素	单调栈(4.7.1)
滑动窗口最大值	单调队列(4.7.2)
静态区间求和	前缀和(4.8.1 / 4.8.2)
区间统一修改	差分(4.8.3)
所有可能 / 列出所有解	回溯(4.9)
局部最优 → 全局最优(难证明)	贪心(4.10)
和为 K 的子数组	HashMap + 前缀和(5.1)
位 1 个数 / 2 的幂 / 只出现一次	位运算(5.2)

七、章节末尾汇总

7.1 模板必备度清单(10 大算法)

#	算法	必背度	高频题号
1	排序	★★★★	LeetCode 215
2	二分	★★★★	LeetCode 33 / 34
3	双指针(对撞/快慢/滑动窗口)	★★★★	LeetCode 15 / 167 / 3



4	BFS / DFS	★★★	LeetCode 102 / 104 / 200
5	DP(背包 / 路径)	★★★	LeetCode 70 / 62 / 198
6	并查集	★★	LeetCode 200 / 547
7	单调栈 / 单调队列	★★	LeetCode 739 / 239
8	前缀和 / 差分	★★	LeetCode 303 / 560
9	回溯	★★★	LeetCode 46 / 78 / 47
10	贪心	★★	LeetCode 55 / 435 / 455

7.2 3 种常用技巧必备度

#	技巧	必背度
1	HashMap + 前缀和	★★★
2	位运算(`n & (n-1)` / 异或)	★★
3	Trie	★★★(进阶,速成可不学)

练习建议

- 把 10 大算法模板各抄 1 遍(不要光看,抄完才记得住)
- 每种算法做 1 道 LeetCode 经典题验证:
- 排序 → LeetCode 215 数组中的第 K 个最大元素(堆排)
- 二分 → LeetCode 33 搜索旋转排序数组
- 双指针 → LeetCode 15 三数之和
- BFS → LeetCode 994 腐烂的橘子
- DFS → LeetCode 200 岛屿数量
- DP → LeetCode 198 打家劫舍
- 并查集 → LeetCode 200 / 547
- 单调栈 → LeetCode 739 每日温度
- 前缀和 → LeetCode 560 和为 K 的子数组
- 回溯 → LeetCode 46 全排列
- 贪心 → LeetCode 55 跳跃游戏

下一步

打开 06_常见坑点速查.md,学 15 大经典坑,Java 刷题救命文档。

